Drift — Decision Log V2 (Claude Code Handoff)

This is the canonical, updated reference for building Drift. Decision Log V1 exists separately and documents the iteration history (ideas considered, rejected, evolved). This doc reflects final decisions only. Figma is the source of truth for layout and content. If this doc describes an element that isn't in the Figma screens, skip it.

Product thesis

"Music tools organize by metadata, but we experience music as feeling." Drift maps songs spatially using audio features so users can explore their library by vibe and build DJ sets through the shape of their taste.

Tech stack

- React + Tailwind + Framer Motion
- React Flow (spatial map, node/wire system)
- Three.js (particle visualizer in Deck View)
- Supabase (persistent storage, feature cache)
- Vercel (deployment)
- SoundNet via RapidAPI (audio feature analysis)
- iTunes Search API (album art)
- Spotify oEmbed (URL-to-name resolution, free, no auth)

Data model (Supabase)

Table: tracks

Column	Type	Notes
id	uuid	Primary key
name	text	Track title
artist	text	Artist name(s)
album	text	Album name (from oEmbed/SoundNet)
album_art_url	text	iTunes Search API result

Column	Type	Notes
bpm	float	Beats per minute (raw value, not normalized)
energy	float	0–100 scale
mood	float	0–100 (valence)
danceability	float	0–100
acousticness	float	0–100
instrumentalness	float	0–100
speechiness	float	0–100
loudness	float	dB value
liveness	float	0–100
key	text	Musical key (e.g., "C#")
camelot	text	Camelot notation (e.g., "3A")
duration	float	Seconds — used for wrong-version detection
popularity	float	0–100 (retained from SoundNet, not displayed in V1)
source	text	"soundnet"
analyzed_at	timestamp	When features were cached
status	text	"analyzed" / "unanalyzed" / "partial"
missing_features	text[]	Array of feature names missing, if partial

Table: playlists

Column	Type	Notes
id	uuid	Primary key
name	text	User-editable, default "Import – [date]"
created_at	timestamp	
user_id	text	"demo" or user identifier

Table: playlist_tracks

Column	Type	Notes
id	uuid	Primary key
playlist_id	uuid	Foreign key → playlists
track_id	uuid	Foreign key → tracks

Table: sets

Column	Type	Notes
id	uuid	Primary key
playlist_id	uuid	Foreign key → playlists
name	text	User-editable set name
created_at	timestamp	
updated_at	timestamp	
user_id	text	Spotify user ID or "demo"

Table: set_tracks

Column	Type	Notes
id	uuid	Primary key
set_id	uuid	Foreign key → sets

Column	Type	Notes
track_id	uuid	
position	int	Order in the chain (1 = head)
is_connected	boolean	True if in active chain, false if orphaned
group_id	text	Orphan group identifier (null if connected)

Table: set_connections

Column	Type	Notes
id	uuid	Primary key
set_id	uuid	Foreign key → sets
source_track_id	uuid	Outgoing socket
target_track_id	uuid	Incoming socket
bpm_delta	float	Computed on connection
key_relationship	text	"same" / "adjacent" / "parallel" / "distant"
compatibility_tier	text	"strong" / "mild" / "weak"

Playlists & Library Model

- One playlist active on map at a time. Switching playlists swaps visible songs.
- Playlist panel shows playlist names, not song lists. The map is the song list.
- Import creates a playlist. "Import more" in panel adds to current or creates new.
- Sets are tied to a parent playlist. Set builder only operates on active playlist.
- No empty map state - users always have songs (demo or import).
- V1: can add songs to a playlist, cannot remove.

API details

SoundNet (primary audio features)

- Provider: RapidAPI
- Cost: ~$10/month, 50k requests
- Lookup: name-based search (artist + title)
- Returns: full feature set including key/Camelot
- CRITICAL: SoundNet returns HTTP 200 on both hits AND misses. Detect misses by checking for `error` key in JSON response body, not HTTP status code.
- Scale: all values 0–100 (except BPM which is raw, and loudness which is dB)
- Cache every result in Supabase. Each track analyzed once.

Spotify oEmbed (URL-to-name resolution)

- Endpoint: `https://open.spotify.com/oembed?url={TRACK_URL}`
- No API key. No auth. No user cap. Public endpoint.
- Returns: JSON with `title` (track name) and `author_name` (artist)
- Purpose: when users paste Spotify URLs instead of "Artist – Title" text, extract track IDs from URLs, call oEmbed to get names, then send names to SoundNet.

iTunes Search API (album art)

- Endpoint: `https://itunes.apple.com/search?term={artist}+{title}&entity=song&limit=1`
- Free. No auth. No key.
- Returns: JSON with `artworkUrl100` (replace "100x100" with "600x600" for higher res)

Import flow — dual format detection

User pastes into the import field. Claude Code detects format:

- Lines starting with `https://open.spotify.com/track/` → extract track ID → call oEmbed → get artist + title → send to SoundNet
- Lines matching "Artist – Title" or "Artist - Title" → send directly to SoundNet
- Mixed formats accepted. One input, two parsers, same downstream pipeline.

Fallback: Exportify

Link provided in import UI for users with large/messy libraries. Exportify gives clean CSV export from Spotify with proper artist/title formatting. This is a user-side workaround, not a system integration.

ReccoBeats

Removed entirely from architecture. Coverage test proved SoundNet handles the library. ReccoBeats couldn't return key data anyway, making it useless for the one feature (Camelot) that justified the paid source. Single-source architecture is the correct call.

Design decisions (updated and complete)

Layout & architecture

#	Decision	Choice	Rejected & why
1	Map layout	Full-width, full-bleed canvas; UI floats on top	Two-column — reduces map space
2	Axis format	Cross/quadrant creating 4 vibe zones	Right-angle; 3D (UX complexity, scope, React Flow 2D conflict)
3	Default axes	Energy × Mood	
4	Axis presets	Vibe (Energy × Mood), Dancefloor (BPM × Danceability), Texture (Acousticness × Energy), Vocal (Instrumentalness × Speechiness)	
5	Mix Ready preset	Shelved to V2	Camelot is circular, doesn't fit continuous-axis model
6	Deck View	Side panel on map, user stays in map context	Separate page — breaks spatial context
7	Three.js visualizer	Panel accent / expandable focus mode; runs off cached feature data, not live audio stream. Reacts fully for all visitors regardless of playback tier.	
8	Left rail	~50px icon rail with Playlists, Set Creation, Explore By + Profile pinned bottom	
9	Panel system	Panels slide from icon rail, one at a time, overlay map (Apple Maps pattern)	

10	Dual-panel state	Playlist left + Deck right accepted as edge case (~30–35% map visible)

Song nodes & zoom

#	Decision	Choice	Rejected & why
11	Zoom tiers	Circle → pill → card. Continuous subtle scaling within circle tier (25–42px), then snap morph to 175px pill at threshold, then snap morph to 300px card at next threshold.	Smooth continuous morph (creates broken mid-transition states)
12	Morph behavior	Snap-triggered, auto-completing ~400ms animation (cubic-bezier 0.16, 1, 0.3, 1). Cannot park mid-transition. Album art is constant anchor through all states.	Hard instant swap (feels disconnected); continuous interpolation (allows broken states)
13	Pill width	175px fixed, universal for all songs	Variable width by name length — distorts spatial reading
14	Card width	300px fixed, universal for all songs	Variable width — same reason
15	Text overflow	Ellipsis truncation. Pill shows shorter text, card shows more due to wider space. Full name visible in Deck View on click.	Wrapping (inconsistent heights)
16	Album art glow	Dominant color extraction via Canvas API (color-thief or vibrant.js). Color derived from album art.	External packages for color
17	Song overlap	True positions always — mapped to a large coordinate space so zooming reveals spatial granularity (Google Maps model). Stack badges reserved for genuinely identical values only. Stack badge for identical-value songs. Popover list on badge click (not fan-out).	Force-directed layout (displaces songs from true positions); fan-out (collides with neighbors on crowded maps)
18	Stack detection	Distance-based: if two songs' centers are within one circle-diameter at current zoom, group into stack. Threshold scales naturally with zoom.	Fixed pixel threshold (doesn't adapt to zoom)

19	Stack popover	Click badge → floating popover list near stack showing all songs with art, name, artist, BPM, key. Click a song to select. Click outside to dismiss. Auto-pans map only when popover would clip viewport edge.	Fan-out (collision risk); side panel (too heavy for this interaction)
20	Axis labels	Axis terminator pills at default zoom; compass widget when zoomed in. Sticky axis pills at viewport edges + zone chip (e.g., "Low energy · Bright mood") when zoomed into a quadrant.	
21	Long track names	Wrap to two lines on cards, making cards taller. Intentional.	
22	Data padding	Map 5%–95% of canvas, not edge-to-edge. Songs with extreme values sit near axis labels, not on top of them.	

Semantic vocabulary (user-facing, not API terms)

Feature	Low pole	High pole
Energy	Chill	Intense
Mood (valence)	Dark	Bright
BPM	Slow	Fast
Danceability	Mellow	Groovy
Acousticness	Electronic	Acoustic
Instrumentalness	Vocal	Instrumental
Speechiness	Melodic	Spoken

Explore By panel

#	Decision	Choice
23	Presets	4 presets: Vibe, Dancefloor, Texture, Vocal. Displayed as 2×2 grid with single compass.

24	Custom axes	Dropdown selectors from full feature list. Selecting a preset fills dropdowns. Touching a dropdown flips to Custom state. Apply button resolves live-vs-applied ambiguity.
25	Preset colors	Cut. Too many color systems competing (accent, wire compatibility, Camelot key colors).
26	Active state	Active-state indicator on selected preset.

Compatibility system

#	Decision	Choice
27	Tiers	Three: strong / mild / weak. Determined by weakest-link rule across BPM + key.
28	BPM thresholds	±6 or less = strong, ±7–15 = mild, ±16+ = weak
29	Camelot thresholds	Same key or adjacent (3A→2A, 4A) = strong; parallel (3A→3B) = mild; everything else = weak
30	Wire colors	Green (strong) / amber (mild) / red (weak)
31	Compatibility card	Fixed corner position. Appears on wire click, disappears on click-elsewhere. Shows: verdict with color, BPM delta, key transition (e.g., "3A → 4A"), relationship type (adjacent/parallel/distant).
32	Camelot key colors	12 positions, programmatically generated in blue/purple/pink/teal range. A and B variants share hue. Green/amber/red avoided (reserved for wire compatibility). Keys shown in gray on map cards and panel rows; teal only in Deck View Camelot circle.

Set builder

#	Decision	Choice	Rejected & why
33	Structure	One head (anchor), one sequential chain. Songs wired one-to-one via sockets. Sequence, not web.	Many-to-many — not how DJ sets work
34	Sockets	Each song has one incoming (hollow) and one outgoing (filled). Four cardinal points (N/E/S/W). Wire takes the edge facing the target. In/out never share an edge.	

35	Cutting a wire	Non-destructive orphaning. Downstream songs dimmed, wires retained between each other, shown in "Disconnected" panel subsection. No confirmation modal.	Destructive delete (loses user's work)
36	Orphan treatment	Dashed border + warm tint. Dim at rest (~40–55% opacity), highlight only on active drag. Distinct from inactive/unconnected song dimming.	
37	Removal from panel	Auto-heals the chain. Reactivation requires deliberate rewiring on map.	
38	Adding songs	Only on the map. Panel corrects and reorders but never adds.	
39	Set creation	Gated at minimum 2 songs.	
40	Wire drag feedback	Dashed white wire while dragging → flashes compatibility color on valid target hover → latches on release → becomes uniform dark wire in chain (flow on) or compatibility-colored wire (flow off). Release on empty = cancel.	
41	Occupied socket	Error icon (✕) at wire-end on occupied-socket hover. Connection doesn't latch.	
42	Head treatment	Halo + size bump (circle zoom); accent border + glow (pill); lighter border + anchor icon (card).	
43	End song	No special treatment. Open outgoing socket marks it.	
44	Head deletion	Non-destructive. Whole chain orphans as one segment. Former song #2 offered as new head (one-tap re-anchor). No confirmation modal — action is recoverable.	Confirmation-gated destructive delete
45	Multiple orphan groups	All shown on map simultaneously. Dim at rest, highlight on active drag. Panel nests under one "Disconnected" section as collapsible sub-groups.	Flat single orphan bucket (destroys wired sub-sequences)
46	Orphan actions	Re-wire to chain = map (add). Dissolve/trim/unlink = panel (correction). Rule: adds on map, corrections in panel.	

47	Panel editing	Reorder existing members → panel (auto-cascades wires). Introduce new song → map.

Flow mode (renamed from Focus mode)

#	Decision	Choice
48	Flow ON	All chained songs full brightness. Everything else ~50% opacity. Orphans always dimmed with dashed border regardless of mode. Wires turn dark grey/black (physical wire aesthetic). Strobe pulse travels head-to-tail at ~10-second intervals. No compatibility wire colors visible.
49	Flow OFF	Full map visible. Wire compatibility colors (green/amber/red) visible. No strobe.
50	Toggle	Pill-style toggle with pulse/activity icon. One icon, two visual states (orange active, gray inactive). Named "Flow."
51	Strobe	Single pulse of light traveling from head through entire chain to tail. Speed scales with chain length so the full chain is traversed in one sweep. ~10-second pause between pulses. CSS animation on SVG path using getPointAtLength().
52	Wire drag in flow mode	The single wire being dragged still flashes compatibility color before latching. Once connected, it becomes the dark uniform wire. Best of both: informed choice at moment of decision, clean view after.

Set builder panel

#	Decision	Choice
53	Visibility	Always open during build mode (not closeable).
54	Content	Set name (inline-editable), numbered list of connected songs, "Disconnected" collapsible subsection, search bar (library-scoped), Save & Complete button (pinned bottom).
55	Song rows	Album art, name, artist, BPM, Camelot key. Head song has green accent treatment. Orphan rows have dashed border + reduced opacity.
56	Search	Relocates to panel in build mode. Finds library songs + locates on map. Copy: "Find a Song on your Map." Result pans/highlights on map.
57	Save button	"Save & Complete" — one button. Persists set to Supabase. Spotify export is V2.

Deck View (bento grid)

#	Decision	Choice
58	Layout hierarchy	Left column (~60% width): large playback disc spanning two rows. Right column: BPM + Camelot, then Danceability gauge. Below: Next Up (left), Compatible Keys (right). Bottom: Energy + Mood sliders.
59	Visualizer hero	Three.js particle simulation. Full width, top of panel. Runs off cached feature data. BPM drives particle speed, energy drives amplitude, mood shifts color.
60	Track info bar	Album art thumbnail, track name, artist, play button. Full width below visualizer.
61	Playback disc	Album art like a vinylwith progress ring (orange accent). Duration counter. Spans the right row
62	BPM circle	Accent 1 ring, large number, "BPM" label.
63	Camelot circle	Accent 2 ring, teal number, "Camelot" label.
65	Compatible keys	Tile showing 3 compatible Camelot keys (adjacent + parallel) as recessed badges. E.g., for 3A: shows 2A, 4A, 3B. "Key unknown" empty state for unanalyzed tracks.
66	Next Up	Track name, artist, BPM delta, key with compatibility color coding. During solo playback: shows nearest compatible track or "Add to a set."
67	Energy + Mood sliders	Dual horizontal sliders with neomorphic recessed treatment. Semantic pole labels (Chill/Intense, Dark/Bright). Switchable presets (Vibe/Dancefloor/Texture/Vocal) that swap both sliders.
68	Dot-matrix BPM meter	Horizontal reactive bar pulsing to BPM. Dot-matrix style (Nothing Phone aesthetic). Sits in the bento grid as a live indicator.
69	Click behavior	Click song on map → opens Deck panel with song paused (not auto-playing).

Aesthetic direction

#	Decision	Choice
71	Map layer	HUD/instrument aesthetic — corner brackets, radar rings, ruler ticks, mono labels, near-black with single accent.
72	Panels & Deck	Bento/gadget aesthetic — tactile rounded modules, soft depth, dot-matrix accent type. Neomorphic recessed surfaces for interactive elements (sliders, search bars, gauges).
73	Color source	Album-art-derived song glows, not chrome decoration.
74	References	Nothing Phone, Inversa, Samur.AI
75	Font	Accent monospace — TBD: B612 Mono, Space Mono, or Chakra Petch. Decide before build.

Playback & Spotify

#	Decision	Choice
76	Playback tiers	30-second previews (iTunes/Deezer) for public visitors. Full playback via Spotify Web Playback SDK for owner + allowlisted Premium accounts (5-user dev mode cap).
77	Visualizer independence	Three.js visualizer runs off cached feature data, not live audio. Reacts fully for all visitors regardless of playback tier.
78	Spotify role	Off critical path. Optional auth layer for playback + future export. Product works fully without it.
79	Distribution	Demo mode with pre-loaded library baked as JSON for public visitors. Live Spotify auth reserved for owner + small circle of allowlisted accounts.
80	Export	V2. V1 has Save & Complete only (persists to Supabase). Spotify playlist export requires OAuth + 5-user cap, not viable for public.

Import flow

#	Decision	Choice
81	Method	Paste-a-tracklist. Accepts both "Artist – Title" text and Spotify URLs. Claude Code detects format and routes accordingly.

82	URL resolution	Spotify URLs → extract track ID → Spotify oEmbed (free, no auth) → get artist + title → SoundNet for features.
83	Welcome screen	Two paths: "Explore the demo library" (primary, hero CTA) + "Paste your tracklist" (secondary, reveals 3-step guide). No account needed for demo.
84	Reconciliation	SoundNet returns single best-guess, no confidence score. Automatic mismatch detection not buildable. Mitigation: require artist + title format; play-to-verify via Deck View.
85	Wrong-version detection	Compare SoundNet duration vs known track length. Flag mismatches >10–15s for user review.
86	Unresolved tracks	Persistent "unresolved" panel. Same component used for import-time reconciliation and ongoing unresolvable tracks.

Storage & persistence

#	Decision	Choice
87	What's persisted	Sets, node arrangements, map positions, cached audio features — all in Supabase tied to user ID or "demo."
88	What's fetched fresh	Album art (iTunes API) — lightweight, no caching needed.
89	Rationale	Sets are core value. Users invest significant time building them. Must survive sessions.

Edge cases (updated)

Multiple orphan groups from chain breaks

Map shows all orphan groups simultaneously, differentiated on-hover (whole segment brightens). Panel nests under one collapsible "Disconnected" section as labeled sub-groups. At rest: dim. During active wire drag: target orphan group highlights as valid target. Orphan actions: re-wire → map (add); dissolve/trim → panel (corrections).

Deleting the head song

Whole chain drops to "Disconnected" as one orphan segment (wires retained, dimmed). Former song #2 offered as new head — one tap to re-anchor. No confirmation modal. Action is non-destructive by design.

Incomplete track data on import

- Full data → plotted normally
- No data → unanalyzed subpanel
- Partial data → plotted on presets where both required axes exist; absent on presets missing a required axis; never plotted at a fake position
- Map-edge count: "N tracks not shown on this view," tappable to reveal which songs and which features are missing

Selecting from a cluster (builder mode)

Click badge → popover list. Clicking a song in the popover wires it to the current tail immediately if a chain exists, or sets it as head if no chain exists. No separate select-then-drag step.

Occupied socket feedback

Three drag states: valid open socket → green highlight; occupied socket → error icon (✕) at wire-end; empty space → neutral trailing wire (cancel on release). Feedback at the wire-end where the user is looking.

Import reconciliation / mismatched songs

No-match → unanalyzed bucket. Wrong matches undetectable (SoundNet single best-guess, no confidence score). Mitigation: artist + title format required; play-to-verify via Deck View.

Unanalyzed/fresh tracks

SoundNet may return 200 OK with no features for very recent releases (~3 weeks old observed). Routes to unanalyzed bucket. Freshness gap, not quality gap. EDM-heavy libraries may see more unanalyzed tracks.

Wrong-version matching

SoundNet can return a different version (e.g., 2:49 edit vs 6:02 original). Detector: compare SoundNet duration vs track length at import. Flag >10–15s delta for user review.

Songs near canvas edge

Data padding: map 5%–95% of canvas. Songs with extreme values sit near axis labels, not on top of them.

Stack popover at viewport edge

Auto-pan map only when popover would clip viewport edge. No pan when space is available. Standard map behavior (Google Maps pattern).

Flow mode with empty chain

Disable Flow toggle until chain has a head. Flow mode with zero songs = everything dimmed = useless.

Deck View during set playback vs solo view

Next Up tile shows specific next song with compatibility data during set playback. During solo browse: shows nearest compatible track or "Add to a set" CTA.

Key unknown state

Tracks without key data (import failures, partial analysis) show "—" in Camelot circle and "Key unknown" in compatible keys tile. These tracks cannot contribute key data to compatibility scoring — compatibility falls to BPM-only assessment.

Scope — V1 vs V2

V1 (ship this)

- Spatial map with 4 presets + custom axes
- Zoom-responsive nodes (circle → pill → card)
- Set builder with node/wire mechanics
- Flow mode with strobe
- Deck View with all tiles
- Paste-a-tracklist import (URLs + text)
- SoundNet audio features + Supabase cache
- 30-second preview playback
- Demo mode with pre-loaded library
- Save & Complete (Supabase persistence)

V2 (shelved)

- Mix Ready preset (Camelot as axis)
- Spotify playlist export
- Spotify OAuth for general users
- Rekordbox XML export
- Cross-playlist sets
- Essentia.js client-side fallback

- Lyrics
- Real-time waveform

Build approach

Vertical slices, not horizontal layers

Get one song on the map with zoom morphing working end-to-end first. Then add the second song with a wire. Then add compatibility. Then Deck View. Each slice is a working demo. If you stop at any point, something is shippable.

What Figma provides

Visual design, layout proportions, colors, type sizes, component structure, style names + descriptions. Claude Code reads this via Figma MCP.

What this document provides

Interaction behavior, state transitions, conditional logic, animation timing, API cascade, compatibility formula, flow mode behavior, zoom morphing triggers — everything the static screens can't show.

Color styles reference (from Figma)

- Accent 1: orange (progress rings, BPM, sockets, strobe)
- Accent 2: blue (energy slider, secondary emphasis)
- Containers/Panels & Toolbars: panel backgrounds
- Containers/Buttons and Cards: interactive tile backgrounds
- Containers/Inset: recessed surfaces (#080808 — slider wells, search bars)
- Canvas background: deepest dark (#0c0c0c)
- Text/Primary: white
- Text/Secondary: gray
- Border: subtle light border
- Icons/Primary, Icons/Secondary
- Wire Compatibility/Strong: green
- Wire Compatibility/Mild: amber
- Wire Compatibility/Weak: red
- Effect styles: Inset, Extrusion, Selected